

python install on vscode

+download from website get latest version

-enable path

+VScode extensions:

-python

-code runner

Run:

pip --version

python --version

Create Project

Open cmd terminal on project folder

Ctrl + ~

Run:

python -m venv venv

.\venv\Scripts\activate

pip install tensorflow matplotlib pandas numpy

\$env:TF_ENABLE_ONEDNN_OPTS=0

Create new file named [numberclassification.py](#)

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dropout,
Flatten, Dense
```

```
import numpy as np
import os
import sys
```

```
import matplotlib.pyplot as plt
import pandas as pd
```

```

# ===== DATA PREPARATION STAGE ===== #

# 1.) Load training and testing images in "coarse" label. "coarse" label
means "types of object" (e.g., fruits, vegetable, etc.)
(a, coarse_TrLabels), (b,coarse_TsLabels) =
keras.datasets.cifar100.load_data(label_mode='coarse') # Tr: Training; Ts:
Testing
print('Coarse Class: {}'.format(np.unique(coarse_TrLabels)))

# 2.) Load training and testing image in "fine" label. "fine" labels
refers to "name of object" (e.g., apple, carrots, etc.)
(fine_train, fine_TrLabels), (fine_test, fine_TsLabels) =
keras.datasets.cifar100.load_data(label_mode='fine')
print('Fine Class for all: {}'.format(np.unique(fine_TrLabels)))

# 3.) Extract all images of a specific coarse class from the TRAINING
DATASET
# The "4" below is the assigned coarse class to "fruits and
vegetables" images.
idx = [] # index holder
for i in range(len(coarse_TrLabels)):
    if coarse_TrLabels[i] == 4: # checks the coarse label of each sample in
the training dataset
        idx.append(i) # append the index of an input image with "fruits and
vegetables" coarse label.

print('Total images with 4 coarse label (fruits and vetegables) from
TRAINING DATASET: {}'.format(len(idx)))
idx = np.array(idx)

# 4.) Extract all image and corresponding "fine" label and store in
train_images, train_labels variable list.
train_images, train_labels = fine_train[idx], fine_TrLabels[idx]
print("Shape of the image training dataset:
{}".format(train_images.shape))
uniq_fineClass = np.unique(train_labels)
print('Fine Class for the extracted training images:
{}'.format(uniq_fineClass))

```

```

# 5.) Extract all images of a specific coarse class from the TESTING
DATASET
# The "4" below is the assigned coarse class to "fruits and
vegetables" images.
idx = []
for i in range(len(coarse_TsLabels)):
    if coarse_TsLabels[i] == 4:
        idx.append(i)

print('Total images with 4 coarse label (fruits and vetegables) from
TESTING DATASET: {}'.format(len(idx)))
idx = np.array(idx)

# 6.) Extract all image and corresponding "fine" label and store in
test_images, test_labels variable list.
test_images, test_labels = fine_test[idx], fine_TsLabels[idx]
print("Shape of the image testing dataset: {}".format(test_images.shape))
uniq_fineClass = np.unique(test_labels)
print('Fine Class for the extracted testing images:
{}'.format(uniq_fineClass))

# 7.) Relabel training and testing dataset to start from zero (0).
for i in range(len(uniq_fineClass)):
    for j in range(len(train_labels)):
        if train_labels[j] == uniq_fineClass[i]:
            train_labels[j] = i

    for j in range(len(test_labels)):
        if test_labels[j] == uniq_fineClass[i]:
            test_labels[j] = i

# Plot few samples from images from the TESTING DATASET
plt.figure(figsize=(10,2))
for i in range(5):
    plt.subplot(1,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(test_images[i], cmap=plt.cm.binary)
    plt.xlabel(test_labels[i])

```

Run using python numberclassification.py on cmd terminal

Add to code:

```
# Build the model
model = tf.keras.Sequential()
# 32 convolution filters used each of size 3x3
model.add(Conv2D(32, kernel_size=(3, 3), activation='relu',
input_shape=(32, 32, 3)))
# choose the best features via pooling
model.add(MaxPooling2D(pool_size=(2, 2)))
# flatten since too many dimensions, we only want a classification output
model.add(Flatten())
# fully connected to get all relevant data
model.add(Dense(128, activation='relu'))
# output a softmax to squash the matrix into output probabilities
model.add(Dense(len(uniq_fineClass), activation='softmax'))

model.summary()

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=[tf.keras.metrics.SparseCategoricalAccuracy()])

metricInfo = model.fit(train_images, train_labels, epochs=10,
validation_split=0.1)

loss = metricInfo.history['loss']
val_loss = metricInfo.history['val_loss']
epochs = range(1, len(loss) + 1)

plt.clf()
plt.plot(epochs, loss, 'g-', label="Training loss")
plt.plot(epochs, val_loss, 'b-', label='Validation loss')
plt.title('Training vs Validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
```

```
plt.show()
```

Run using python numberclassification.py on cmd terminal

Add to code:

```
# Test the model
str_class = ['apple', 'mushroom', 'orange', 'pear', 'bell pepper']

print(test_images.shape)
print("Class in the testing image: {}".format(np.unique(test_labels)))
test_loss, test_acc = model.evaluate(test_images, test_labels)
print('Total number of testing image: {}'.format(len(test_images)))
print('Test accuracy:', test_acc)

# Another way to test using "prediction" method
classification = model.predict(test_images)
print('\nDisplaying prediction of the first test input image:
{}'.format(classification[0]))

# get the index of the maximum probability in the classification[0] result
max_prob_idx = np.argmax(classification[0])
print('Predicted class: {}--{}'.format(max_prob_idx,
str_class[max_prob_idx]))
idx = test_labels[0]
print('True class: {} -- {}'.format(idx[0], str_class[idx[0]]))
```

Run using python numberclassification.py on cmd terminal

Create new file named sinewave.py

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dropout,
Flatten, Dense

import numpy as np
```

```

import os
import sys
import math

import matplotlib.pyplot as plt

SAMPLES = 1000
SEED = 1337

np.random.seed(SEED)
tf.random.set_seed(SEED)

x_values = np.random.uniform(low=0, high=2*math.pi, size= SAMPLES)
#np.random.shuffle(x_values)
y_values = np.sin(x_values)

TRAIN_SPLIT = int(0.60*SAMPLES)
TEST_SPLIT = int(0.2*SAMPLES + TRAIN_SPLIT)

y_noise = y_values.copy()
y_noise += 0.1*np.random.randn(*y_values.shape)

x_train, x_validate, x_test = np.split(x_values, [TRAIN_SPLIT,
TEST_SPLIT])
y_train, y_validate, y_test = np.split(y_noise, [TRAIN_SPLIT, TEST_SPLIT])

y_clean = y_values[TEST_SPLIT:]

plt.plot(x_values, y_values, 'b.')
plt.plot(x_train, y_train, 'g.')
plt.show()
print('{}'.format(x_train.shape))

```

Run using python sinewave.py on cmd terminal

Add to code:

```
# Build model=====
model = tf.keras.Sequential()
model.add(Dense(10, activation='relu', input_shape=(1,)))
model.add(Dense(50, activation='relu'))
model.add(Dense(10, activation='relu'))
model.add(Dense(1))

model.compile(optimizer='rmsprop', loss='mse', metrics=['mae'])
model.summary()

# Train the model
metricInfo = model.fit(x_train, y_train, epochs=50,
                        validation_data=(x_validate, y_validate))

loss = metricInfo.history['loss']
val_loss = metricInfo.history['val_loss']

epochs = range(1, len(loss) + 1)

plt.plot(epochs, loss, 'g', label='Training Loss')
plt.plot(epochs, val_loss, 'b', label='Validation Loss')
plt.title('Training and Validation Loss')
plt.legend()
plt.show()
```

Run using python sinewave.py on cmd terminal

Add to code:

```
predictions = model.predict(x_test)
plt.clf()
plt.plot(x_test, y_clean, 'b.', label='Observation')
plt.plot(x_test, predictions, 'r*', label='Predictions')
plt.legend()
plt.show()

model.save('baseModel.h5')
```